

ASIGNATURA: Ingeniería de Software 2
PERÍODO ACADÉMICO: 2026-0
FECHA: 19/02/2026
TIEMPO: 85 minutos

NOTA

EXAMEN ESCRITO Nº 02 (12 puntos)

CÓDIGO	APELLIDOS Y NOMBRES	SECCIÓN

ANTES DE INICIAR LA EVALUACIÓN DEBE LEER LAS INSTRUCCIONES

Si la evaluación indica cargar algún archivo en la computadora, recuerde que es responsabilidad del estudiante hacerlo en el tiempo establecido y con las instrucciones dadas. Debe indicar el número de la misma en el recuadro siguiente:

INSTRUCCIONES GENERALES:

- La evaluación consta de “2” preguntas, cuyo puntaje está indicado en cada una de ellas.
- El procedimiento, el orden, la claridad de las respuestas y el uso apropiado del lenguaje (notaciones, símbolos y unidades), serán considerados como criterios de calificación.
- Puede consultar material del curso o de soporte.
- **Leer detenidamente las situaciones que ocasionarán la anulación de la evaluación, que se encuentran a continuación.**

SITUACIONES QUE OCASIONARÁN LA ANULACIÓN DE LA EVALUACIÓN:

- Tener acceso a internet o utilizar herramientas de IA generativas.
- Mantener prendidos teléfonos celulares, relojes smart, así como cualquier otro medio o dispositivo electrónico de comunicación.
- Compartir o intercambiar hojas, tablas, cualquier material impreso, dispositivo electrónico, durante el desarrollo de la evaluación.
- Conversar durante el desarrollo de la evaluación.

Los profesores de la asignatura

FIRMA DEL ALUMNO
(LEYÓ LAS INSTRUCCIONES)

PREGUNTA 1 (6 puntos)

La semana pasada la NASA anunció el descubrimiento de 7 planetas del tamaño de la Tierra orbitando la estrella Trappist-1, con potencial de albergar vida. A partir de esto, se impulsan proyectos universitarios para muestrear porciones de la galaxia y estimar la probabilidad de encontrar sistemas con condiciones para la vida.

Tu equipo implementa un software que analiza un **cuadrante estelar** representado por coordenadas **(X, Y, Z)** y calcula una probabilidad **$p \in [0,1]$** . Según el valor de p , se deben generar alertas:

- Si $p \leq 0.5 \rightarrow$ **SIN ALERTA**
- Si $0.5 < p < 0.7 \rightarrow$ notificar a **investigadores de la universidad**
- Si $p \geq 0.7 \rightarrow$ notificar a **científicos de la NASA**

Se te entrega un proyecto Maven con la clase GalaxySurveyor, que contiene **cuatro métodos** de análisis:

- analyzeA(Quadrant q)
- analyzeB(Quadrant q)
- analyzeC(Quadrant q)
- analyzeD(Quadrant q)

Cada uno implementa el análisis con reglas diferentes y, por lo tanto, tienen distinta complejidad ciclomatica.

P1A) Elija uno de los métodos de análisis y redacte una especificación semiformal (estilo IEEE / especificación estructurada) que incluya como mínimo (3 puntos):

- Propósito
- Entradas
- Salidas
- Precondiciones
- Postcondiciones
- Excepciones / errores esperados

La especificación debe de guardarla y enviarla por Blackboard con el nombre p1_especificacion.txt

P1B) Para el método de análisis B, implemente pruebas unitarias utilizando Junit. Debe realizar pruebas para todos los caminos independientes (según la complejidad ciclomática de este método). Debe enviar todo el proyecto + las pruebas unitarias en un .zip con el nombre "GalaxySurveyor" (3 puntos)

PREGUNTA 2 (6 puntos)

La empresa ha implementado un módulo interno de Helpdesk que procesa tickets de soporte y calcula:

1. Costo estimado del ticket (en USD y convertido a PEN).
2. SLA objetivo (minutos máximos para atender).
3. Escalamiento si el ticket excede el SLA (notificaciones internas).

El módulo funciona “en producción”, pero el CTO ha detectado que el código se está volviendo difícil de mantener y muy costoso de extender (por ejemplo, agregar un nuevo tipo de cliente o severidad implica cambios en muchas partes).

Se te entrega un proyecto Maven con pruebas existentes. Tu tarea es mejorar el diseño manteniendo el comportamiento.

Restricciones

- Debes **mantener el comportamiento observable** (si cambias el diseño, las reglas deben seguir dando el mismo resultado).
- Debe seguir pasando: `mvn test`
- Puedes agregar pruebas nuevas, pero **no debes eliminar** las existentes.
- No es necesario agregar frameworks extra: usa Java + JUnit5.

P2A) Detectar 3 antipatroneos o smells. Debe crear el archivo llamado `p2_smells.txt` donde detalle el nombre del smell o antipatrón, el lugar del código en el que sucede el antipatrón o smell, y como lo planea solucionar. (3 puntos)

P2B) En base a las soluciones planteadas en el punto anterior, debe realizarlas en el código entregado y enviar el programa completo zipeado corregido por Blackboard. (3 puntos)